Brève introduction à la programmation en Python

Mozhdeh Shahbazi, PhD, PEng
Ressources naturelles Canada
Centre canadien de cartographie et d'observation de la Terre

Introduction

- La programmation consiste à concevoir une séquence d'instructions afin de résoudre un problème.

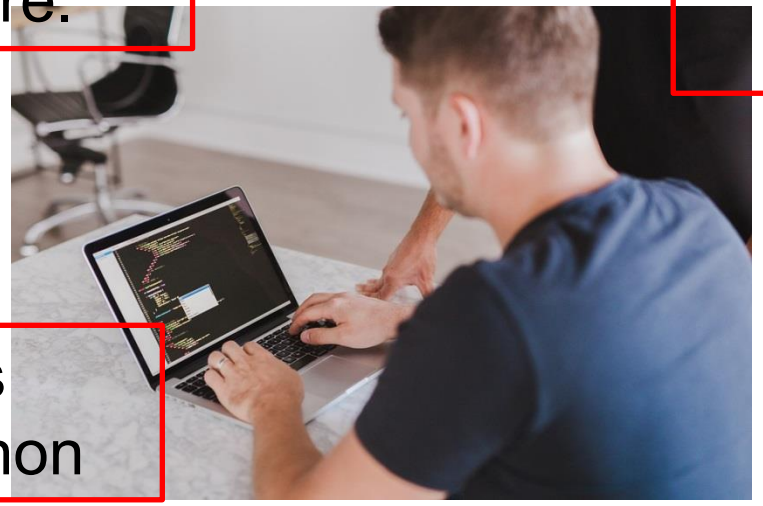
- Défi : rendre ces instructions compréhensibles par le compilateur ou l'interpréteur (un logiciel capable de traduire des instructions humaines en langage machine).

- Solution: les instructions doivent être écrites sous forme de code source, en respectant les règles et la syntaxe imposées par le langage de programmation.

Introduction

Analyser comment résoudre le problème. Concevoir les instructions afin de le résoudre.

Déboguer et corriger le programme



Programmer ces instructions en Python

Exécuter le programme

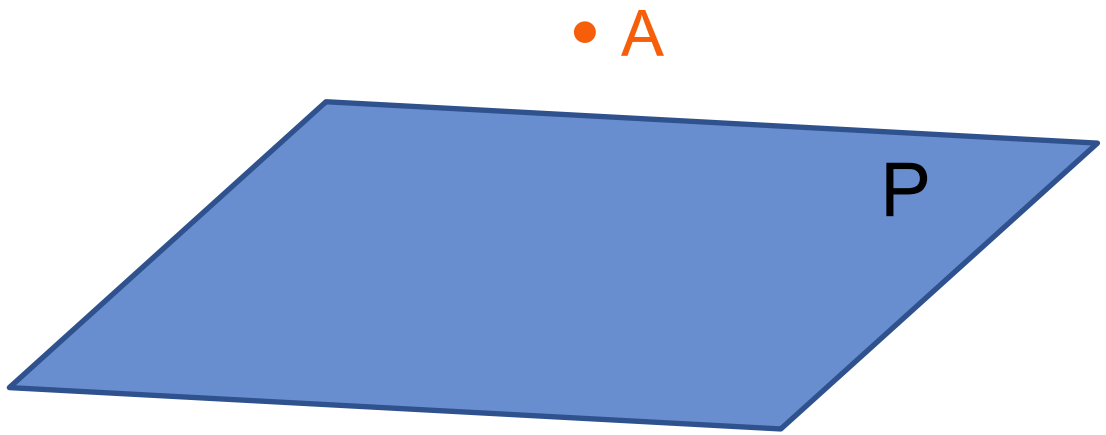
Image source: pixabay.com

Exemple de code source

```
import math

def take_sqrt():
    input_as_string = input("enter a number: ")
    input_as_float = float(input_as_string)
    input_sqrt = math.sqrt(input_as_float)

    print("Square root of your number is:", input_sqrt)
    return input_sqrt

if __name__ == '__main__':
    take_sqrt()
```

Programmation

- La programmation se compose d'une série d'instructions.
- Les instructions sont une série d'opérations appliquées à des données et qui peuvent, optionnellement, retourner une valeur.
- Exemple
  - Problème : Le point A est-il situé sur le plan P ?

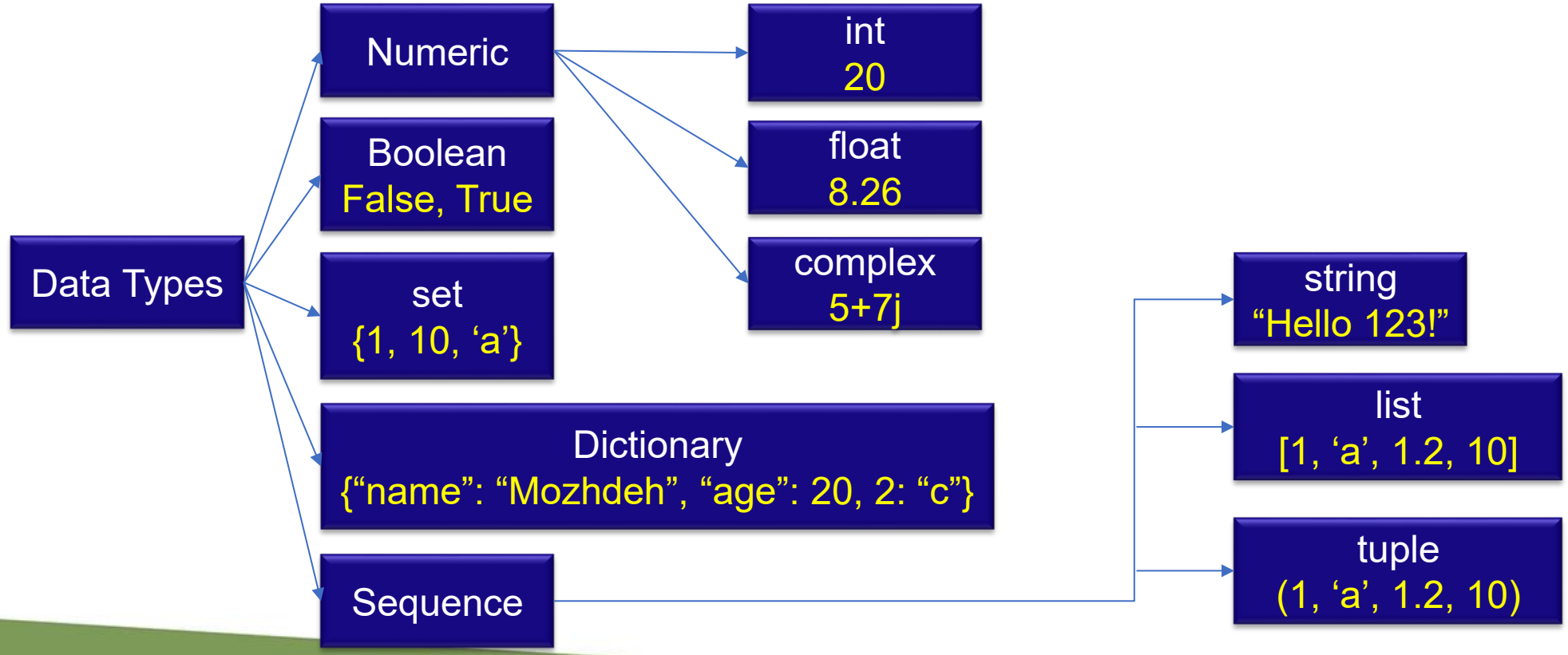


Programmation

- Exemple
  - Problème : Le point A est-il situé sur le plan P ?
  - Instructions pour la solution :
    - Prendre les coordonnées du point A
    - Prendre les paramètres du plan P
    - Calculer la distance séparant A du plan P
    - Si la distance est égale à zéro
      - Le point se trouve sur le plan : retourner oui
    - Sinon
      - Le point n'est pas sur le plan : retourner non

Variables

- Les variables sont nécessaires pour stocker les données
- Une variable possède un nom, une valeur et un type



Variables

- Nombres

```
a = 100
b = 2.5
c = 1 + 4j
d = float('3.44')
e = int('10')
```

- Booléen

```
status = False
is_correct = True
```

Variables

- Ensembles

```
this_set = {"apple", "banana", "cherry", 10, 11, 12.5}

print(len(this_set))

another_set = set(("apple", "banana", "cherry"))
```

- Dictionnaires

```
a = {'myset': {1,2}, 'name': 'Mozhdeh', 100: 'hello'}
print(a.values())
print(a.keys())
print(a['name'])
```

Variables

- String

```
# This is comment in Python
message1 = "Hello"  # String
message2 = 'Hello in single quote'  # This is also a String.

message3 = message1 + " World..!"  # The returned data type is also a string

# Type will return the data type.
print(type(message2))  # <type 'str'>.
```

Variables

- Liste et tuple

```
a = [1, 10, 'a232', 1]
c = a[2]
d = a[-1]
print(a[1:3])
a[0] = 'hey'
a.append(-10.5)
a.extend([1001, 102.1])
a.append(['b', 'c'])
e = (1, 10, 'a232', 1)
e[0] = 1 #This is not allowed
```

Opérateurs

- Un code source est constitué d'instructions
- Une instruction contient une ou plusieurs expressions
- Les expressions réalisent leur tâche à l'aide d'opérateurs
- Quelques opérateurs :
  - Opérateur d'affectation (=)
  - Opérateurs arithmétiques (+, -, *, /, % ,//,**)
  - Opérateurs d'affectation composée (+=, -=, *=, /=, %=)
  - Opérateurs relationnels et de comparaison (==, !=, >, <, >=, <=)
  - Opérateurs logiques sous forme de mots-clés (and, or, not, is)
- Priorité des opérateurs : [*, /, %] → [+, -] → [<, <=, >, <=] → [==, !=] → [and, or] → =

Structures de contrôle

- **Branchement conditionnel** (structures de décision)
  Exécutent certaines parties du code selon une condition (ex : if, else).
- **Structures itératives**
  Permettent de répéter une séquence d'instructions (ex : for, while).

Structures de contrôle

- Les décisions sont prises à l'aide des opérateurs relationnels et logiques
- Formes de base :
  - if
  - if/else
  - if/elif/else imbriqués

```
x = 100
if x > 0:
    print('x is a positive number')
elif x < 0:
    print('x is a negative number')
else:
    print('x is zero!')
```

Structures de contrôle

- Formes de base des structures itératives (boucles) :
  - While
  - For
- Il existe deux façons d'influencer l'exécution des boucles en Python :
  - **continue**: Retourne immédiatement au début de la boucle, effectue la mise à jour de la boucle (dans le cas d'une boucle for), puis réévalue la condition d'arrêt
  - **break**: Sort immédiatement de la boucle et exécute la première instruction qui suit la boucle

Structures de contrôle

```
students = ['Mozhdeh', 'Sarah', 'Tom']
students_ids = {}
for student in students:
    a = input("Enter ID of {}; Q to exit; C if you don't know their ID: ".format(student))
    if a == ‘Q':
        break
    if a == ‘C':
        continue
    students_ids[student] = int(a)

for student in students_ids.keys():
    iD = students_ids[student]
    print(student, ":", iD)
```

Les fonctions – Pourquoi les utiliser?

- Réutilisation : Que se passe-t-il si nous devons répéter plusieurs fois un même ensemble d'instructions ? Doit-on les copier plusieurs fois ?
  - Le programme devient long et difficile à lire ou à suivre.
  - Il devient aussi trop compliqué à maintenir et à mettre à jour.
- Solution : Les fonctions permettent de réutiliser un bloc de code sans avoir à le répéter.
- Une fonction se caractérise par :
  - Son corps (le code exécuté)
  - Un identifiant (nom de la fonction)
  - (optionnel) Des paramètres
  - (optionnel) Un type et une valeur de retour
- En Python, une fonction a deux « aspects » : sa définition (implémentation) et son appel (utilisation).

Apprendre Python avec l'aide de l'intelligence artificielle

« Apprends-moi Python en me guidant à travers un petit projet.

Découpe le projet en étapes.

Assure-toi que différents types de variables sont introduits grâce à cet exemple de projet, et que les concepts de fonctions (au début) puis de classes (plus tard) soient également abordés.

Pour chaque étape : explique le concept, montre un exemple de code, puis donne-moi un petit exercice à réaliser afin de valider l'étape, demande-moi de saisir mon code pour cela, vérifie mon code et indique-moi s'il est correct.

Ne passe pas à l'étape suivante tant que je n'ai pas terminé l'étape en cours. »

© His Majesty the King in Right of Canada, as represented by the Minister of Natural Resources, 2026